



APRENDIZAGEM DE MÁQUINA

Otávio Calaça Xavier



INF

INSTITUTO DE
INFORMÁTICA



UFG

UNIVERSIDADE
FEDERAL DE GOIÁS



PODER JUDICIÁRIO

Tribunal de Justiça do Estado de Goiás



Programa de
Residência em TI

Aula 7

Aprendizado por Reforço

INF

INSTITUTO DE
INFORMÁTICA



UFMG
UNIVERSIDADE
FEDERAL DE GOIÁS



PODER JUDICIÁRIO

Tribunal de Justiça do Estado de Goiás



Programa de
Residência em TI

Objetivos da Aula

- Compreender a dinâmica agente-ambiente
- Entender estados, ações e recompensas
- Introduzir MDP, função de valor e política
- Apresentar algoritmo Q-Learning
- Mostrar aplicação prática em Python

O que é Aprendizado por Reforço?

- Paradigma baseado em **interação**
- O **agente** aprende por **tentativa e erro**
- Objetivo: maximizar **recompensa acumulada**
- Modelo inspirado em controle e psicologia
- Útil para decisões **sequenciais**

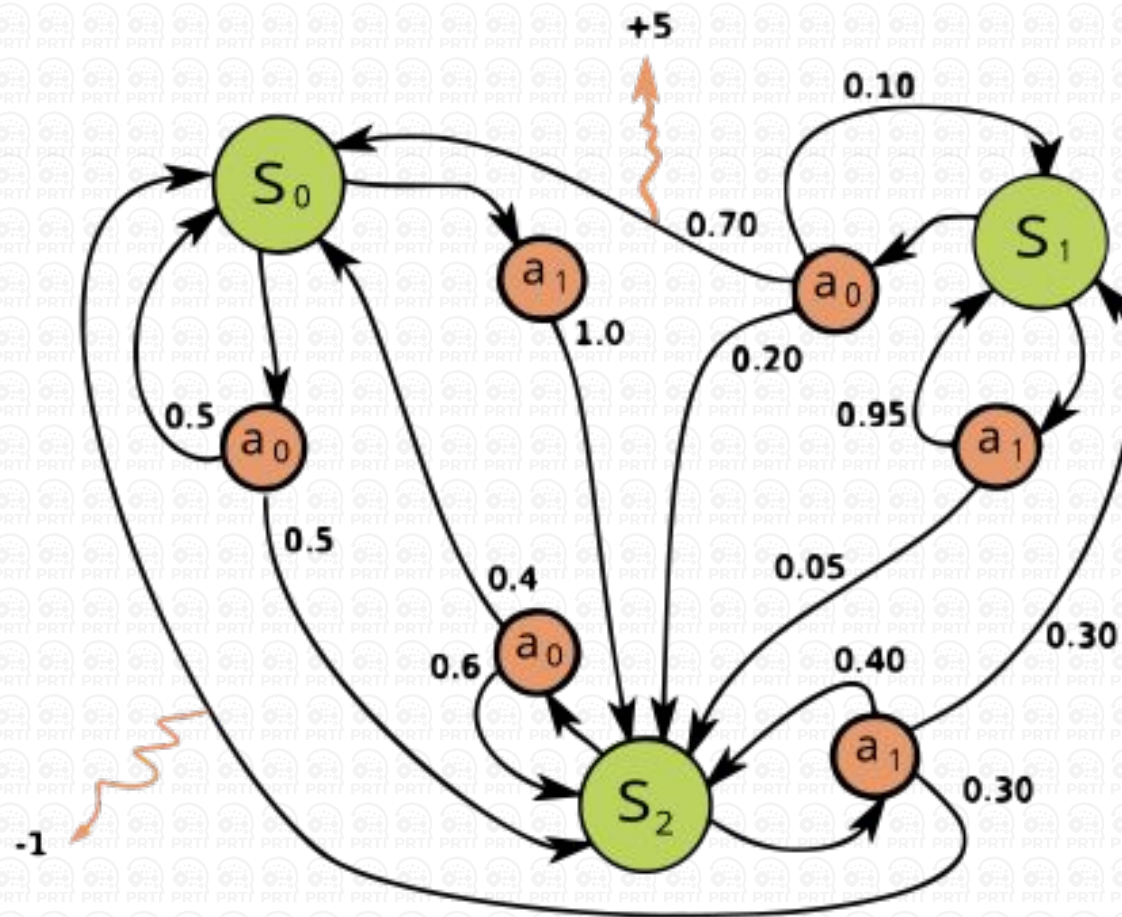
Componentes Básicos

- **Ambiente:** onde o agente atua
- **Agente:** quem toma ações
- **Estado (s):** situação atual
- **Ação (a):** decisão tomada
- **Recompensa (r):** feedback imediato
- **Política (π):** estratégia do agente

Processo de Decisão de Markov (MDP)

- O que é um MDP?
 - Um MDP modela problemas de decisão sequencial sob incerteza.
 - É composto por **estados** (S) e **ações** (A) que o agente pode tomar.
 - As ações produzem transições com **probabilidade**: $P(s' | s, a)$.
 - Cada transição gera uma **recompensa**: $R(s, a, s')$.
 - O ambiente obedece à **propriedade de Markov** (o futuro depende só do estado atual).
 - Objetivo: encontrar a **política ótima** π^* que maximiza o retorno esperado.

Processo de Decisão de Markov (MDP)



Processo de Decisão de Markov (MDP)

- **Estrutura Formal de um Processo de Decisão de Markov**
 - Um MDP é definido pelo conjunto $\langle \mathbf{S}, \mathbf{A}, \mathbf{P}, \mathbf{R}, \gamma \rangle$.
 - $\mathbf{S}$: conjunto de estados possíveis.
 - $\mathbf{A}$: conjunto de ações disponíveis em cada estado.
 - $P(s'|s,a)$: probabilidade de transição para s' ao tomar ação a .
 - $R(s,a,s')$: recompensa imediata ao fazer a transição.
 - γ (gamma): fator de desconto que controla o peso do futuro.

Processo de Decisão de Markov (MDP)

- A Propriedade de Markov

- O futuro depende **somente** do estado atual.
- A trajetória passada não influencia a decisão.
- O estado atual contém toda a informação necessária.
- Simplifica a modelagem e o aprendizado.
- Permite usar função-valor e Q-learning.
- É base para quase todas as técnicas de RL.

Processo de Decisão de Markov (MDP)

- **Exemplo de MDP em um Cenário do TJ**

- **Estado (s):** situação da fila (ex: nº processos urgentes, carga do gabinete).
- **Ações (a):** priorizar, redistribuir, adiar, normalizar fluxos.
- **Transições:** como a fila muda após cada decisão.
- **Recompensa:** redução do tempo de tramitação / cumprimento de metas.
- **Política:** regra que escolhe a melhor ação no estado atual.
- **Objetivo:** maximizar eficiência e reduzir backlog.

Funções de Valor

- $V(s)$: valor de estar no estado s
- $Q(s,a)$: valor de tomar ação a em s
- Define qualidade das decisões
- Base da atualização Q-Learning
- Ajuda a prever retorno futuro
- Política segue máximos de Q

Exploração vs Exploração

- Dilema central do Aprendizado por Reforço.
- **Exploração**: tentar ações novas para descobrir oportunidades.
- **Exploração**: usar o que já sabe para maximizar recompensa agora.
- Estratégia clássica: ϵ -greedy
- → com prob. ϵ explora, com prob. $1-\epsilon$ explota.
- ϵ alto = mais testes; ϵ baixo = comportamento mais estável.
- Controle de ϵ influencia velocidade e qualidade do aprendizado.

Q-Learning: Como o Agente Aprende?

- **Aprende $Q^*(s,a)$:** estima o valor de cada ação em cada estado.
- Baseado na **Equação de Bellman**: atualiza o valor usando recompensa + melhor valor futuro.
- **Off-policy**: aprende a melhor política mesmo tomando ações de outra política (ϵ -greedy).
- **Não precisa conhecer o ambiente**: aprende somente observando experiências (amostras).
- **Atualiza iterativamente**: cada interação melhora a estimativa da Q-table.
- **Política final**: escolher sempre a ação com maior valor Q em cada estado.

Q-Learning: Como o Agente Aprende?

- Baseado na **Equação de Bellman**: atualiza o valor usando recompensa + melhor valor futuro.

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Significados:

- r = recompensa imediata
- γ = peso do futuro (0 a 1)
- $\max Q(s', a')$ = melhor valor no próximo estado
- α = taxa de aprendizado

Q-Learning (Pseudocódigo)

- Inicialize tabela $Q(s,a)$
- Para cada episódio:
 - Comece no estado inicial
 - Escolha ação via ϵ -greedy
 - Observe recompensa e próximo estado
 - Atualize Q :
$$Q \leftarrow Q + \alpha(r + \gamma \max_{a'} Q' - Q)^{**}$$
 - Atualize estado
- Política final = $\text{argmax}(Q)$

Exemplos do Mundo Real

- Robôs e carros autônomos
- Sistemas de recomendação adaptativos
- Agentes conversacionais dinâmicos
- Otimização de filas e recursos
- Alocação e priorização de processos
- Automatização de rotinas jurídicas

Exemplo em Código

- Colab:

https://colab.research.google.com/drive/1tagD3ZsRJXa1O75T_UOI6CrQzSjflo5m?usp=sharing